

Top Product Companies – 100 Interview Questions (Full Answers)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 17 Aug 2026 – 14 Feb 2027 (Days 1–182)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

100 questions actually asked at Google, Meta, Amazon, Microsoft, Uber, Atlassian, Adobe, Salesforce, Flipkart, Stripe, PhonePe, Razorpay.

Each answer is what a hire-level candidate says aloud (2–3 minutes). Spell terms in full on first use.

How to use: 2 questions/day from the tier you are in. Cover the question, then hide the answer and repeat.

Q	Topic	Typical companies
1-15	JavaScript	Google, Meta, Uber, Atlassian, Adobe
16-28	React / frontend	Meta, Atlassian, Uber, Flipkart
29-40	Node.js / backend	Amazon, Uber, Adobe, Razorpay
41-48	TypeScript / Next.js	Atlassian, Vercel-style FE, Adobe
49-58	Databases / Structured Query Language	Amazon, Microsoft, Uber, Adobe
59-68	Data Structures and Algorithms (verbal)	All
69-78	Low Level Design / machine coding	Amazon, Flipkart, PhonePe, Uber
79-90	High Level Design (product)	Google L5, Meta, Uber, Flipkart
91-95	Computer science fundamentals	Microsoft, Amazon, Goldman
96-100	Behavioral / hiring manager	All (Amazon heaviest)

Module banks at the end of each theory PDF are now **100 questions each** as well. This file is the **cross-cutting** set — one place for a full product-company loop.

JavaScript (1-15)

Question 1: Explain the JavaScript event loop, microtasks, and macrotasks. (Google, Meta, Uber)

Answer: JavaScript runs on one call stack. Synchronous code runs first. When the stack is empty, the engine drains the **microtask** queue completely (`Promise.then` , `queueMicrotask` , `MutationObserver`) before taking **one macrotask** (`setTimeout` , `setInterval` , I/O, user events). That is why `Promise.resolve().then(...)` logs before `setTimeout(0)` . In Node.js there are extra phases (timers → pending → poll → check → close) and `process.nextTick` runs before Promises. Interviewers follow up with a 6-line snippet — walk the output, do not recite a blog.

Question 2: What is a closure, and where have you used one in production? (Amazon, Salesforce)

Answer: A closure is a function that keeps access to variables from the lexical scope where it was created, after that outer function has returned. Production uses: private counters, memoization caches, React event handlers that capture props, and debounce timers. Pitfall: a `var` loop shared across timeouts; fix with `let` or an immediately invoked function. Closures can leak memory if a long-lived callback holds a large object — null the reference or abort the listener.

Question 3: How does `this` work, including arrow functions, `call`, `apply`, and `bind`? (Atlassian, Adobe)

Answer: For a normal function, `this` is the caller: `obj.method()` → `obj`; a bare call is `undefined` in strict mode. Arrow functions have no `this`; they inherit from the enclosing scope — so do not put arrows on `prototype`. `call`/`apply` invoke now (`apply` takes an array of args). `bind` returns a new function with `this` locked. Follow-up they love: implement `Function.prototype.bind`.

Question 4: What will `typeof null` and `typeof []` return, and why is that a problem? (Microsoft, Flipkart)

Answer: `typeof null` is `"object"` (a 1995 bug still in the language). `typeof []` is also `"object"`. Distinguish with `Array.isArray`, `value === null`, or `Object.prototype.toString.call(value)`. Never use `typeof` alone to detect “a real object.”

Question 5: Debounce vs throttle – when does a product company want each? (Flipkart, Uber, Amazon)

Answer: **Debounce** waits until events pause (search box, resize) — one Application Programming Interface (API) call after typing stops. **Throttle** allows at most one run per window (scroll handler, button spam). For search: debounce 200–300 ms + `AbortController` so a slow old response cannot overwrite a newer query.

Question 6: How do `Promise.all`, `allSettled`, `race`, and `any` differ? (Google, Meta)

Answer: `all` fails fast on first reject — use when every request must succeed. `allSettled` waits for all and returns `{status, value|reason}` — use for dashboards. `race` settles with the first settle (timeouts). `any` resolves with the first fulfill and ignores rejects until all fail. Say which you pick for “fetch 10 product cards.”

Question 7: Explain prototypal inheritance vs `class`. (Google, Microsoft)

Answer: Objects delegate to a prototype chain (`[[Prototype]]`). `class` / `extends` / `super` is syntax over the same chain. `prototype` lives on the constructor; the instance's internal slot points at that object. You should still be able to set up inheritance with `Object.create` without `class`.

Question 8: What is the Temporal Dead Zone? (Meta, Stripe)

Answer: For `let` / `const`, the binding exists from the start of the block but cannot be read until the declaration line — that gap is the Temporal Dead Zone and throws `ReferenceError`. It exists so you cannot silently use an uninitialized binding the way `var` hoisting allowed (`undefined`).

Question 9: How would you deep-clone an object, and what breaks? (Amazon, Adobe)

Answer: `structuredClone` handles most built-ins (Date, Map, circular in modern engines). `JSON.parse(JSON.stringify)` drops `undefined`, functions, `Date` (becomes string), and `Map`. Libraries like Lodash `cloneDeep` if you must. Never roll a naive recursive clone without a `WeakMap` for cycles.

Question 10: What is the difference between `==` and `===`, and when is coercion dangerous? (All screens)

Answer: `===` is same-type, no coerce. `==` coerces (`0 == ""` is true, `null == undefined` is true). Product code uses `===` except the intentional `== null` check for both `null` and

`undefined`. Interviewers show `[] == ![]` to see if you panic or explain `ToPrimitive`.

Question 11: How does garbage collection work, and what causes a leak in the browser? (Google FE, Meta)

Answer: Mark-and-sweep: reachability from roots (stack, globals). Leaks: detached Document Object Model (DOM) nodes still referenced, forgotten `setInterval`, closures holding large arrays, uncleared event listeners. DevTools heap snapshot: compare before/after navigation.

Question 12: Explain generators and iterators. Where are they used? (Google, Atlassian)

Answer: An iterator is `{ next() → { value, done } }`. A generator (`function*`) pauses with `yield` and implements that protocol. Used for lazy sequences, Redux-saga-style flows, and `for...of` custom collections. `Symbol.iterator` makes an object iterable.

Question 13: What is a Proxy, and when would you use one? (Adobe, some Google)

Answer: `new Proxy(target, handler)` intercepts get/set/has/apply. Uses: reactive state (Vue 3), validation, default values, API logging. Cost: slower than a plain object; do not Proxy a hot path without measuring.

Question 14: `event.target` vs `event.currentTarget`, and what is delegation? (Meta, Flipkart)

Answer: `target` is the deepest node clicked; `currentTarget` is the node with the listener. Delegation: one listener on a parent, branch on `target.closest('[data-id]')`. You avoid thousands of listeners on a list and handle rows added later.

Question 15: How do you cancel an in-flight `fetch` when the user types a new query? (Uber, Atlassian)

Answer: Create an `AbortController` per request; `signal` goes to `fetch`. On the next keystroke, `abort()` the previous controller, then start a new one. Handle `AbortError` as non-

fatal. Combine with debounce. This is the autocomplete machine-coding follow-up.

React / frontend (16-28)

Question 16: What triggers a React re-render, and how do you stop wasted ones? (Meta, Flipkart)

Answer: A re-render happens when state or props change (or a parent re-renders). Fix waste: split state, `React.memo` on pure children, stable callbacks (`useCallback`) only when a memoized child needs them, do not put new object/array literals in props every render. Profile with React DevTools before sprinkling memo.

Question 17: `useEffect` vs `useLayoutEffect` vs `useInsertionEffect` . (Meta, Adobe)

Answer: `useEffect` runs after paint — data fetch, subscriptions. `useLayoutEffect` runs after DOM update, before paint — measure layout, avoid flicker. `useInsertionEffect` is for Cascading Style Sheets-in-JavaScript libraries injecting styles before layout. Server Components cannot use any of these the same way — effects are client-only.

Question 18: Why is the `key` prop required in lists, and why is the index a bad key? (All FE)

Answer: Keys tell React which child is which across renders. Index keys break when you insert/reorder/filter — state sticks to the wrong row (inputs swap). Use a stable business id. Follow-up: keys must be unique among siblings, not globally.

Question 19: Controlled vs uncontrolled inputs. When is each correct? (Atlassian, Uber)

Answer: Controlled: React state is the source of truth (`value` + `onChange`) — validation, disable submit, format as you type. Uncontrolled: `defaultValue` + `ref` — simple forms, file inputs, less re-renders. Machine coding (autocomplete) is almost always controlled.

Question 20: How does reconciliation / the virtual DOM work? Is it always faster? (Google, Microsoft)

Answer: React builds a tree of objects describing the UI, diffs it with the previous tree, and applies a minimal DOM update. It is not “always faster than vanilla DOM” — the win is predictable updates at scale. Keys, bail-outs (`memo`), and avoiding huge trees matter more than the virtual DOM slogan.

Question 21: Explain Context. When should you not use it? (Meta, Adobe)

Answer: Context passes data through the tree without prop drilling. Any consumer re-renders when the provided value identity changes. Do not put a fast-changing value (mouse position, every keystroke) on a wide Context — split contexts or use a store (Zustand/Redux). Provide a stable object (`useMemo`) if the value is a bag of fields.

Question 22: What is hydration, and what causes a mismatch? (Next.js / Meta / Adobe)

Answer: Hydration attaches event handlers to server-rendered HyperText Markup Language. A mismatch means the server HTML and the first client render differ — `Date.now()` , `window` , `Math.random()` , invalid HTML, or browser-only APIs in the render path. Fix: render a placeholder on the server, compute the real value in `useEffect` .

Question 23: How do you optimize Largest Contentful Paint, Interaction to Next Paint, and Cumulative Layout Shift? (Google FE, Adobe)

Answer: Largest Contentful Paint: size the hero image, CDN, do not hide it behind JavaScript. Interaction to Next Paint: break long tasks, debounce input, avoid huge synchronous renders. Cumulative Layout Shift: width/height on images, no late ads without reserved space, `size-adjust` on fonts. Name the metric, then one change you would make on ShopLite.

Question 24: How would you implement infinite scroll without duplicate pages? (Meta, Uber)

Answer: `IntersectionObserver` on a sentinel. Keep `page` and `loading` in refs/state so a double fire cannot fetch twice. Deduplicate by id in a `Set`. Abort on unmount. Virtualize (`react-window`) if the list is thousands of rows.

Question 25: Error boundaries – what do they catch and miss? (Atlassian, Microsoft)

Answer: Class `componentDidCatch` / `getDerivedStateFromError` catch render errors in the subtree. They do **not** catch event-handler errors, async errors, or the boundary's own errors. Wrap routes; show a fallback; log to your monitor. For async, use try/catch and local error state.

Question 26: Client Components vs Server Components in Next.js. (Adobe, Atlassian)

Answer: Server Components render on the server, can read the database, ship zero client JavaScript for that subtree. Client Components (`"use client"`) use state, effects, and browser APIs. Push interactivity to the leaves. Do not fetch in a Client Component what a Server Component could have loaded.

Question 27: How do you prevent prop drilling without overusing Context? (Flipkart, Uber)

Answer: Compose children (`<Layout sidebar={...}>`), lift state only as high as needed, use a lightweight store for cross-cutting auth/theme, or component injection. Prop drilling of 2–3 levels is fine; Context is for truly global, slow-changing data.

Question 28: Build autocomplete in 90 minutes – what is the rubric? (Atlassian, Uber, Razorpay)

Answer: Debounce, abort in-flight fetch, keyboard (arrows, Enter, Escape), `role="listbox"` + `aria-activedescendant`, highlight match, empty/loading/error states, cache prefix → results.

They fail you for no debounce or `div` + `onClick` with no keyboard. Details: PDF #15 §13 and PDF #21 §6.

Node.js / backend (29-40)

Question 29: How does the Node.js event loop handle Central Processing Unit-heavy work? (Amazon, Uber)

Answer: One JavaScript thread. CPU work (big JSON parse, image resize, crypto) blocks every request. Move it to `worker_threads`, a child process, or a queue + worker service. I/O (database, network) is fine on the event loop. Follow-up: clustering (`cluster` or multiple containers) uses more cores for *connections*, not for one fat synchronous function.

Question 30: Middleware order in Express – how do you structure auth and errors? (Adobe, Razorpay)

Answer: `helmet` → request id → JSON parser → route → `requireAuth` → handler. Error middleware is `(err, req, res, next)` and must be last. Async handlers must `next(err)` or wrap with a helper — unhandled promise rejections crash or hang. Never catch-and-swallow.

Question 31: JSON Web Token vs session cookies. When do you revoke access? (Amazon, Flipkart)

Answer: JSON Web Token: stateless, good for APIs and multiple services; hard to revoke before expiry unless you keep a denylist or use short access + rotating refresh tokens. Sessions: server stores id, easy logout. Product default today: `httpOnly` Secure cookie, short access token, refresh rotation, reuse detection.

Question 32: How do you make a payment or “create order” endpoint idempotent? (Stripe, PhonePe, Razorpay)

Answer: Client sends `Idempotency-Key`. Server stores key → result. Same key + same body returns the first result. Different body with same key is 409. Unique constraint on `payment_id`.

This is the payment High Level Design follow-up — they will ask what happens if the webhook fires twice.

Question 33: What is backpressure in Node streams? (Adobe, Uber)

Answer: A fast readable and a slow writable. If you ignore `write()` returning `false` and skip `drain`, memory grows. `pipeline()` / `finished()` handle this. Use streams for file upload and large exports, not `fs.readFile` of a 2 GB file.

Question 34: How do you do graceful shutdown? (Amazon, Microsoft)

Answer: On `SIGTERM`, stop taking new connections, finish in-flight requests (timeout 10–30 s), close the database pool, then `process.exit`. Kubernetes sends `SIGTERM` before kill. If you exit immediately, users get 502s.

Question 35: Structured logging vs `console.log`. What do you log? (All senior)

Answer: JSON logs with `requestId`, user id (not secrets), latency, status. Levels: error for 5xx, warn for 4xx spikes, info for business events. Never log tokens or card numbers. Correlate with traces (OpenTelemetry).

Question 36: Rate limiting – token bucket vs sliding window. (Amazon, Uber, Cloudflare-style)

Answer: Token bucket: burst then steady rate; good for APIs. Sliding window: count events in the last N seconds (Redis sorted set or `INCR+EXPIRE`). Per user and per Internet Protocol. Distributed: Redis, not in-process memory, or each replica has its own limit.

Question 37: How do you prevent Structured Query Language injection and Cross-Site Scripting? (All)

Answer: Parameterized queries / Prisma / bind variables — never string-concatenate SQL. Output-encode HTML; Content-Security-Policy; httpOnly cookies. Validate with Zod at the edge. Object-level authorization on every request (not only “is logged in”).

Question 38: N+1 queries – how do you find and fix them? (Microsoft, Uber)

Answer: Loop that hits the database per item. Fix: `IN` / join / DataLoader batch, or GraphQL DataLoader. Find with logging (`DEBUG=prisma*`) or APM. Follow-up: a covering index so the batch query is cheap.

Question 39: Clustering vs a process manager vs containers. (Adobe, Amazon)

Answer: `cluster` forks workers on one machine. Process managers (PM2) restart on crash. Production at product companies: multiple container replicas behind a load balancer, not PM2 on a pet virtual machine. Horizontal Pod Autoscaler is the Kubernetes version — mention only if they ask deploy.

Question 40: How would you design the TaskFlow notification send path? (Uber, Flipkart)

Answer: API writes `notifications` row + enqueue job (queue). Workers send email/push/SMS. Retry with backoff. Idempotency key = `notificationId+channel` . Do not send email inside the HTTP request. Deep dive: PDF #10 notification design.

TypeScript / Next.js (41-48)

Question 41: `interface` vs `type` , and when do you use `unknown` ? (Atlassian, Adobe)

Answer: Both describe shapes; `interface` can merge (declaration merging), `type` can do unions/intersections more freely. Prefer `unknown` over `any` — you must narrow before use. `never` is what remains in an exhaustive switch.

Question 42: Explain generics with a real API helper. (Microsoft, Stripe)

Answer: `async function getJson<T>(url: string): Promise<T>` so the caller gets typed data. Constraints (`T extends { id: string }`) document what you need. Do not over-generic a one-off function.

Question 43: What does `strict` in TypeScript buy you? (All TS rounds)

Answer: `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`. It catches the bugs product companies care about: “string | undefined passed to a function that wants string.” Your Catalog API must compile with `strict: true` (Phase 2 gate).

Question 44: How do discriminated unions help reducer/state machines? (Atlassian)

Answer: A shared `kind` or `type` field lets TypeScript narrow: `if (event.type === "loaded")` then `event.data` exists. This is how you type a vending-machine or order-state Low Level Design in TypeScript.

Question 45: Next.js App Router: where does data fetching live? (Adobe, Meta-adjacent FE)

Answer: Server Components / `fetch` on the server with cache options. Route Handlers for mutations you do not want in the page. Client fetch only for user-specific interactive data. Know `revalidate` / `cache: 'no-store'` at a high level.

Question 46: What is a Server Action, and what is the security risk? (Adobe)

Answer: A function you can call from a form that runs on the server. Risk: it is an implicit public endpoint — authenticate and authorize inside, validate input, do not trust hidden fields.

Question 47: How do you type `process.env` safely? (All Node TS)

Answer: Zod-parse env at boot; fail fast if `DATABASE_URL` is missing. Do not sprinkle

`process.env.X!` with assertions.

Question 48: Utility types you must know: `Pick`, `Omit`, `Partial`, `Record`.
(Microsoft, Atlassian)

Answer: `Pick<User, "id" | "email">` for public DTOs. `Omit` to strip password. `Partial` for patches. `Record<Id, User>` for maps. Interviewers ask you to type an update-user payload in 60 seconds.

Databases / SQL (49-58)

Question 49: When do you pick PostgreSQL vs MongoDB vs Redis? (Amazon, Uber, Flipkart)

Answer: PostgreSQL: money, orders, joins, transactions. MongoDB: flexible documents, high write, fewer joins. Redis: cache, sessions, rate limits, locks — not the source of truth for orders. ShopLite orders = PostgreSQL; TaskFlow comments can be Mongo; cart/session = Redis.

Question 50: Explain isolation levels and a dirty read. (Microsoft, Amazon, Goldman)

Answer: Read uncommitted can dirty-read. Read committed (Postgres default) sees only committed data. Repeatable read / snapshot avoids non-repeatable reads. Serializable prevents phantoms — use for seat booking or say `SELECT FOR UPDATE`. Product talk: checkout uses a transaction; like-count does not.

Question 51: How do B-tree indexes help, and when do they hurt? (Amazon, Adobe)

Answer: They speed `WHERE`, `JOIN`, `ORDER BY` on that column. They hurt writes (every insert updates the index) and they are useless if the predicate is not selective or you wrapped the column in a function (`WHERE LOWER(email)=`). Always `EXPLAIN ANALYZE`.

Question 52: Write the idea of “employees earning more than their manager.” (Amazon OA / Microsoft)

Answer: Self-join: `e.salary > m.salary` on `e.manager_id = m.id`. Timed SQL set: PDF #21 §4. Say the join first, then filters.

Question 53: What is a covering index? (Microsoft, Uber)

Answer: An index that contains every column the query needs, so the engine never visits the heap/table. Example: `(user_id, created_at, status)` for “latest orders for user.”

Question 54: Optimistic vs pessimistic locking for inventory. (Flipkart, BookMyShow, Amazon)

Answer: Pessimistic: `SELECT ... FOR UPDATE` — wait, then update. Optimistic: version column, update only if version matches, retry on 0 rows. Seats/flash sale: pessimistic or a Redis lock + database constraint. Never check-then-set without a lock.

Question 55: What is connection pooling and why does it matter in Node? (Adobe, Amazon)

Answer: Opening a Transmission Control Protocol connection + auth per query is slow and will exhaust the database. A pool (PgBouncer or Prisma’s pool) reuses connections. Size it: `replicas × (cores)` — too many pools from 50 Node processes kill PostgreSQL.

Question 56: Replication lag – what does the user see? (Meta, Uber)

Answer: Writes go to primary; reads from replicas can be stale. After “post tweet,” read-your-writes from primary or a cache you just wrote. Feed for others can be replica/eventual.

Question 57: How do you migrate a large table without locking production? (Amazon, Microsoft)

Answer: Expand-contract: add nullable column, backfill in batches, dual-write, switch reads, drop old. Never `ALTER` a 200-million-row table in one transaction in peak hours.

Question 58: Redis cache-aside vs write-through. (All HLD)

Answer: Cache-aside: app reads cache, miss → database → fill cache. Write-through: write cache and database together. Aside is the default; accept a miss after write unless you invalidate/update the key on write.

Data Structures and Algorithms – verbal (59-68)

Question 59: How do you talk through a coding problem in a Google/Meta screen?

Answer: Restate + constraints + two examples (empty, duplicate) → brute force + complexity → better idea → code with names → trace → invite follow-up (stream, memory, threads). Silence over 20 seconds is a no-hire. Script: PDF #8 \$15 and PDF #20 \$4.3.

Question 60: When do you use two pointers vs a sliding window vs a hash map? (All)

Answer: Two pointers: sorted array, pairs, palindrome. Sliding window: contiguous subarray/substring with a constraint. Hash map: frequency, “have I seen this,” subarray sum. Say the pattern name in the first minute.

Question 61: Merge intervals – algorithm and follow-up. (Meta, Amazon, Uber)

Answer: Sort by start, scan, merge if `cur.start <= last.end`. Follow-up: Meeting Rooms II —

min-heap of end times. Calendar with recurrences: expand then merge. Code: PDF #8 §17.

Question 62: How do you find the Kth largest in a stream? (Google, Uber)

Answer: Size-K min-heap: if new > min, pop and push. $O(\log K)$ per insert. Median of a stream: two heaps (max + min). This is LC 215 / 295 — they will ask space if K is huge.

Question 63: Graph: number of islands vs course schedule. (Amazon, Microsoft)

Answer: Islands: grid Depth-First Search / Breadth-First Search, mark visited, count components. Course schedule: directed graph, cycle = impossible → topological sort or DFS colors. Draw the graph first.

Question 64: Dynamic programming – how do you know it is DP in 60 seconds? (Google, Microsoft)

Answer: Optimal substructure + overlapping subproblems. Ask: “If I knew the answer for $n-1$ / a prefix, could I build n ?” Then define `dp[i]`, base, transition, walk an example. Coin change, house robber, longest increasing subsequence are the warm-ups.

Question 65: Lowest Common Ancestor in a binary tree vs a Binary Search Tree. (Meta, Amazon)

Answer: Binary Search Tree: walk from root, both nodes left or right, else this is the ancestor. General tree: recurse; if left and right both find a node, root is the ancestor. Null-safe.

Question 66: Design Least Recently Used cache – structures and complexity. (Amazon, Google, Flipkart)

Answer: Hash map key → node, doubly linked list for order. `get` / `put` $O(1)$: move node to head, evict tail. They will ask you to code it (LC 146) and then “what if distributed?” (Redis + TTL is a different problem).

Question 67: Binary search on the answer – example. (Google, Uber)

Answer: When the predicate “can we finish with capacity mid?” is monotonic. Koko bananas, ship packages: search mid in `[max(piles), sum]`. Prove monotonicity aloud.

Question 68: Follow-up: the array does not fit in memory. What do you say? (Google L5)

Answer: External sort, two-pass, map-reduce, or stream with a heap. Do not freeze. This follow-up is the L4 vs L5 difference (PDF #20 §4.3).

Low Level Design / machine coding (69-78)**Question 69: How do you start a 90-minute machine coding round? (Flipkart, Amazon, PhonePe)**

Answer: 10 min clarify (actors, must-have vs later). Write interfaces/enums first. Happy path. Then edge cases. Think aloud. Last 5 min: one test and “what I would add.” Rubric: PDF #15.

Question 70: Parking lot – classes and extension. (Amazon, Flipkart)

Answer: `Vehicle` (types), `Spot`, `Floor`, `Ticket`, `ParkingLot` with strategy for assignment. New vehicle type = new subclass + spot compatibility, not a switch explosion. SOLID: single reason to change per class.

Question 71: Rate limiter – token bucket in memory, then distributed. (Uber, Amazon, Cloudflare)

Answer: In memory: tokens, refill timestamp, mutex. Distributed: Redis Lua INCR+EXPIRE or token bucket keys per user. They ask what happens with 3 API boxes — in-process memory is wrong.

Question 72: Splitwise – how do you settle? (Flipkart, PhonePe)

Answer: Net balance per user. Greedy: match max debtor with max creditor until zero. Equal/exact/percent splits. Do not store every pairwise IOU if you can net. Extensibility: new split type = strategy.

Question 73: BookMyShow seats – how do you not double-sell? (Flipkart, Ticketmaster-style)

Answer: Hold with expiry (8 min), `SELECT FOR UPDATE` or unique `(event, seat)` + status. Payment webhook marks sold, idempotent on `payment_id`. Sweeper releases holds. Kafka is not the inventory source of truth (PDF #10 §18).

Question 74: Vending machine – which pattern? (Amazon, Microsoft)

Answer: State: Idle, HasMoney, Dispensing, OutOfStock. Transitions are methods on the state. Interviewers watch whether you avoid a 40-branch if-else.

Question 75: Elevator – what do they actually grade? (Amazon, Uber)

Answer: Direction, pending floors, multiple cars (assign nearest going the right way). Working dispatch beats a perfect simulation of physics. Time-box a single car first.

Question 76: Frontend machine coding vs backend – what changes? (Atlassian, Uber)

Answer: Same 90-min clock. Rubric adds accessibility, debounce, and UI states. Autocomplete / kanban / comments. PDF #15 §13.

Question 77: How do you show SOLID in 2 minutes without a lecture? (All LLD)

Answer: One sentence each with *your* class: Single — `Ticket` does not send email. Open/closed — new vehicle without editing `ParkingLot`. Liskov — `ElectricCar` can fill a

`CarSpot`. Interface segregation — do not force `Fly()` on `Car`. Dependency inversion — `ParkingLot` depends on `SpotAllocator` interface.

Question 78: Evaluation: what is a no-hire in machine coding? (Flipkart bar)

Answer: No running happy path, god class, silent failures, cannot add a type without rewriting everything, mute for 10 minutes. Hire: running demo, clear types, one extension story.

High Level Design – product (79–90)

Question 79: How do you open a 45-minute High Level Design? (Google, Meta, Uber)

Answer: User + one metric + Minimum Viable Product vs v2 (3 min). Requirements. Application Programming Interface JSON. Data model + keys. Happy path. Hottest queries per second. User-visible failure. Do not open with Kafka. Framework: PDF #20 \$5.1.

Question 80: Design a URL shortener. (All – first HLD)

Answer: `POST /links` → code; `GET /:code` → 302. Base62 or hash. PostgreSQL (`code`, `url`, `expires`). Redis cache for hot codes. 100M writes/month is tiny; reads dominate. Collision: retry. Custom aliases: unique constraint.

Question 81: News feed – fan-out on write vs read. (Meta, Twitter/X)

Answer: Write: push post id to each follower's cache (fast read, celebrity explodes). Read: pull from followees at read time (slow for big graphs). Hybrid: fan-out normal users, pull celebrities. Metric: p99 feed load.

Question 82: Chat / WhatsApp – delivery and order. (Meta, Uber, Google)

Answer: WebSocket gateway, sticky session or connection map in Redis. Messages durable in a store, at-least-once + client idempotency. Receipts: sent / delivered / read. Group chat: fan-out

to members via a queue. Multi-device: sync mailbox per user.

Question 83: Notification system. (Amazon, Uber, Flipkart)

Answer: Producer → queue → workers per channel (push, email, Short Message Service). Template service. Preferences table. Retry + dead letter. Do not block the originating API.

Question 84: Uber matching (basic). (Uber, Ola, Swiggy)

Answer: Drivers in geo index (geohash / hex). Rider request → nearby available → offer → accept. Surge = demand/supply per cell. Consistency: a driver cannot take two trips — lock the driver id. Deep dive: PDF #10 Uber section.

Question 85: YouTube playback. (Google, Netflix-style)

Answer: Upload → object store → transcode jobs → Adaptive Bitrate renditions → Content Delivery Network. Play path should not hit the application server for bytes. Recommendations are offline. Metric: start-play p99. PDF #10 §15.

Question 86: Google Drive / Dropbox sync. (Google, Dropbox)

Answer: Chunk + hash, metadata database, conflict = version + notify. Client uploads missing chunks, commits version. PDF #10 §16.

Question 87: Typeahead / search box. (Google, Meta, Atlassian)

Answer: Prefix index, p99 < 50 ms, cache hot prefixes, offline popularity job. Typos are not a full corpus edit-distance on the request. PDF #10 §17.

Question 88: Payments – exactly-once-ish. (Stripe, PhonePe, Razorpay)

Answer: Ledger rows, idempotency keys, webhook signature verify, reconciliation job. Saga if you also reserve inventory. Strong consistency on money. PDF #10 payment + Ticketmaster.

Question 89: What breaks at 10× traffic? (All HLD follow-ups)

Answer: Name the hottest path (usually reads). Cache, read replicas, then shard on a key you can query. Queues absorb write spikes. Show numbers: DAU × actions / 86400 × peak 2.5.

Question 90: Consistency cheat-sheet in product language. (Google, Meta)

Answer: Ledger/seats/payments = strong. Like counts = eventual. Feed = read-your-writes for me, eventual for others. Chat = at-least-once + de-dupe. Typeahead = best effort. Table: PDF #20 §5.6.

Computer science fundamentals (91-95)**Question 91: Process vs thread vs the Node event loop. (Microsoft, Amazon)**

Answer: Process = isolated memory. Threads share a heap and need locks. Node is one thread + event loop for I/O; CPU work needs workers. Deadlock: four conditions; prevent with lock order (seat then payment). PDF #20 §7.

Question 92: Transmission Control Protocol vs User Datagram Protocol, and why Hypertext Transfer Protocol uses the first. (Microsoft, Adobe)

Answer: Transmission Control Protocol: reliable, ordered, handshake. User Datagram Protocol: low latency, no retry (video chunks, games). Hypertext Transfer Protocol and Hypertext Transfer Protocol Secure run on Transmission Control Protocol. Transport Layer Security encrypts; often terminated at the load balancer.

Question 93: What happens when you type a URL and press Enter? (Google FE, Amazon)

Answer: Domain Name System → Transmission Control Protocol + Transport Layer Security → Hypertext Transfer Protocol request → HTML parse → Cascading Style Sheets Object Model → render tree → layout → paint. JavaScript can block parse. PDF #21 §5.

Question 94: ACID in one minute with a checkout example. (Amazon, Goldman)

Answer: Atomic: stock decrement and order insert both commit or neither. Consistent: stock never negative (constraint). Isolated: two checkouts do not oversell (lock or serializable). Durable: after commit, a crash does not lose the order (write-ahead log).

Question 95: CAP during a partition – bank vs social feed. (Google, Amazon)

Answer: Network split: you choose consistency or availability. Bank ledger: Consistency-Partition (refuse the write). Feed like-count: Availability-Partition (show a stale number). PACELC: even without partition you trade latency vs consistency.

Behavioral / hiring manager (96–100)

Question 96: Tell me about yourself (80 LPA / 1 Cr switch). (All S2 recruiter + HM)

Answer: 60–90 seconds: current role → one metric (latency, revenue, reliability) → why this company/team → fullstack TypeScript + product systems. No life story. Script structure: PDF #16.

Question 97: A production incident you owned. (Amazon Dive Deep / Bias for Action)

Answer: Situation: checkout errors 5%. Task: restore checkout. Action: logs → Redis race → atomic transaction + test + alert. Result: 5% → 0%, 30 days clean. Use *I*, not *we*. Numbers required. Amazon: name the Leadership Principle.

Question 98: You disagreed with your manager. (Amazon Have Backbone, Google collaboration)

Answer: Data, not ego. You stated the risk, offered a smaller experiment, committed once decided (Disagree and Commit). Result and what you would repeat. Never trash the manager.

Question 99: Why this company, and where in two years? (Hiring manager – all 100 companies)

Answer: One product surface you care about (payments, feed, collab) + how your TaskFlow/ShopLite maps. Two years: deeper IC scope (bigger surface, on-call, mentoring), not “become a manager” unless true. Team-match pitch: PDF #22 §4.9.

Question 100: How do you handle an underperforming peer? (Googleyness, Meta, Amazon Hire and Develop)

Answer: Private, specific examples, offer help, document, escalate to the manager if it affects the user. You do not gossip and you do not silently carry their work forever. Close with the outcome for the *user* and the team.

How to finish this file

- ☐ 100 questions answered aloud once
- ☐ 20 answers recorded (phone)
- ☐ Company you are applying to: highlight the rows in the table at the top
- ☐ Same week: 2 questions from the **100-question module bank** at the end of that week's theory PDF

Related: PDF #17 (tiered 140), PDF #20 (loops), PDF #22 (which company uses which stage).